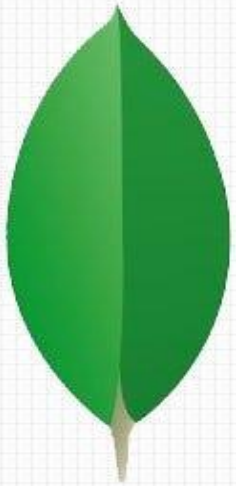
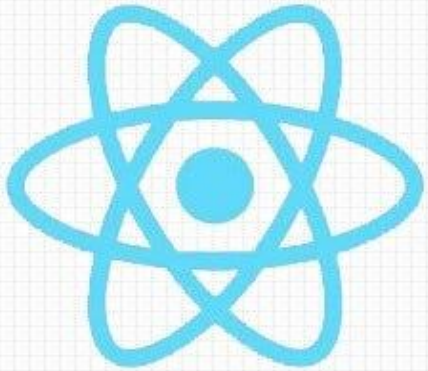
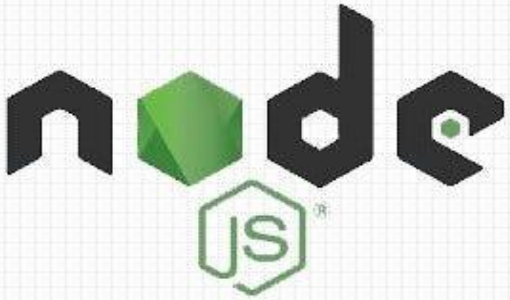
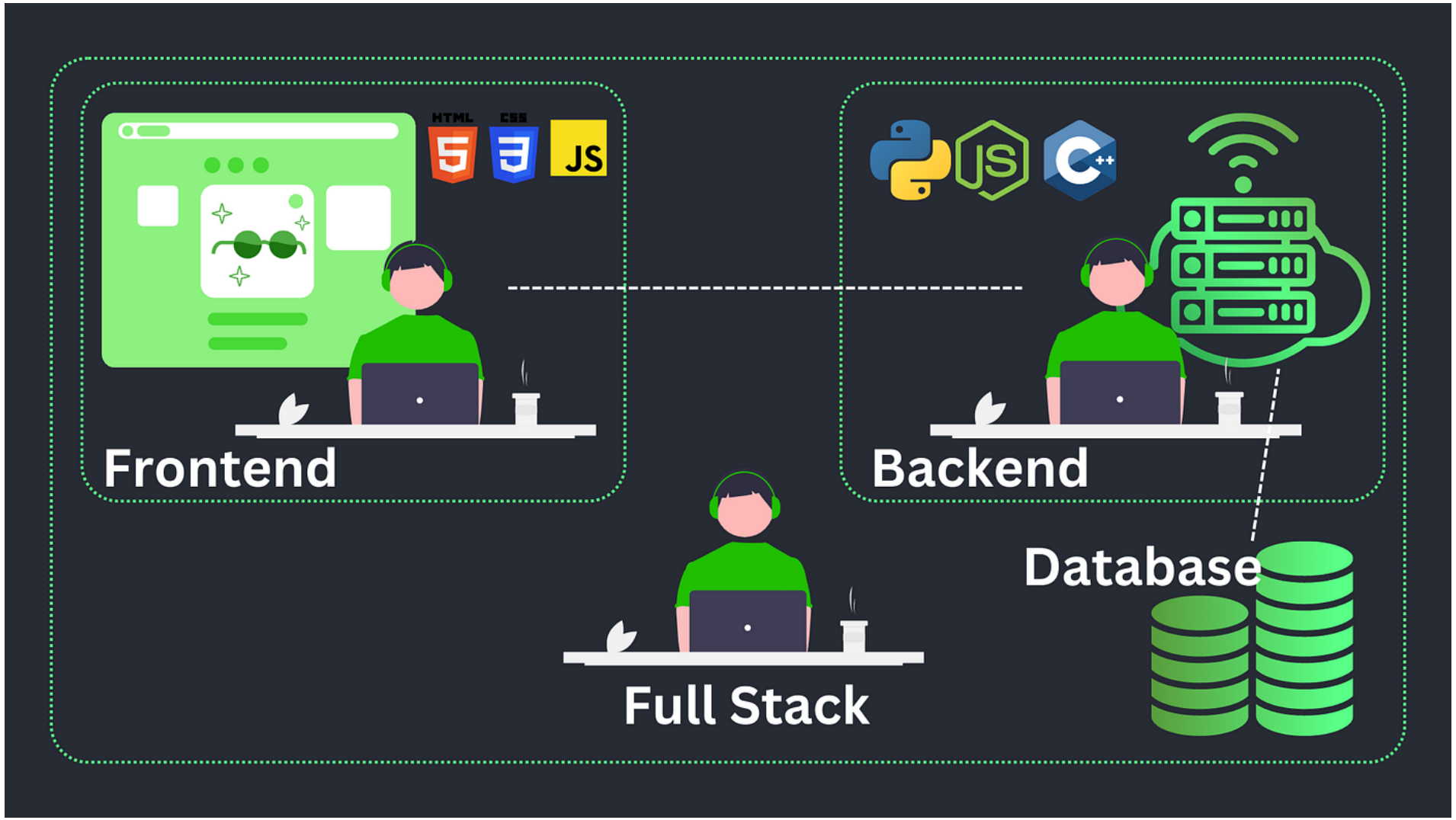
MERN
Stack Development



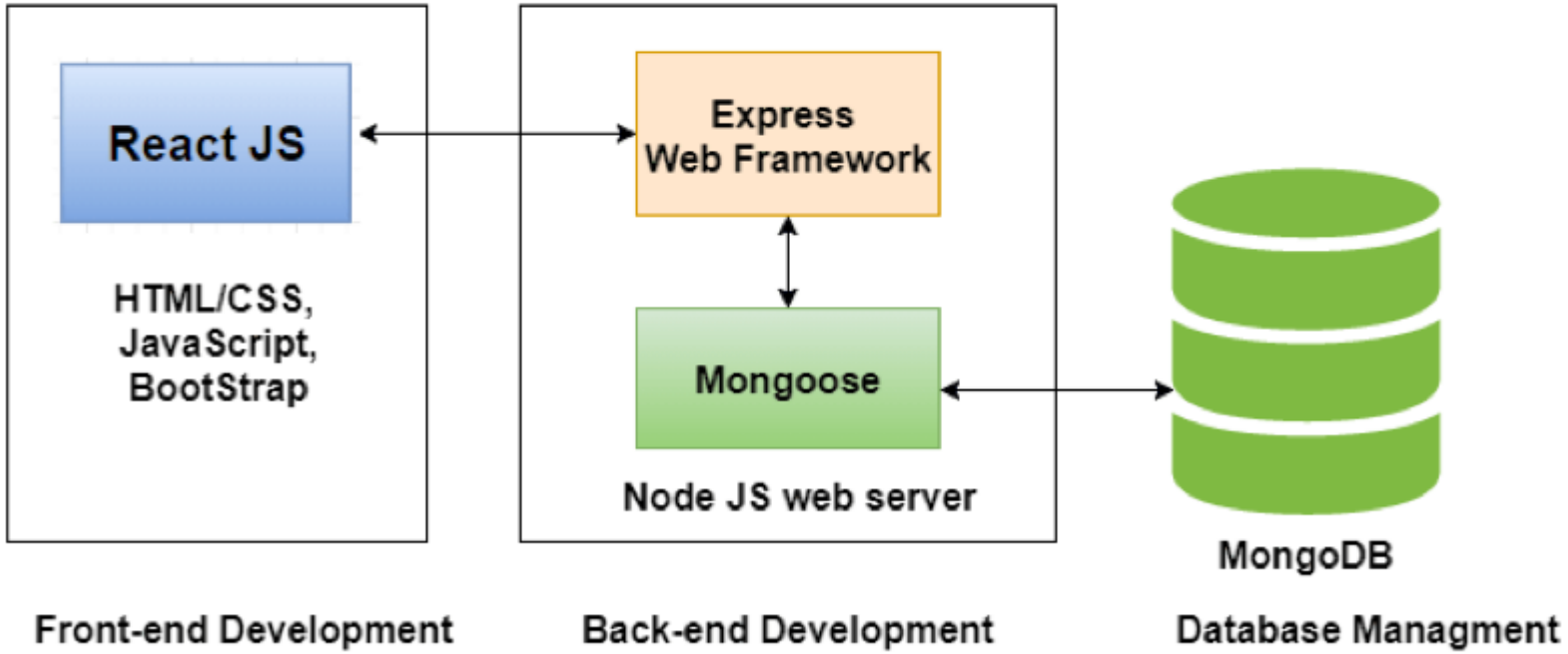



M E R N

Raja Meenakshi
Corporate Trainer

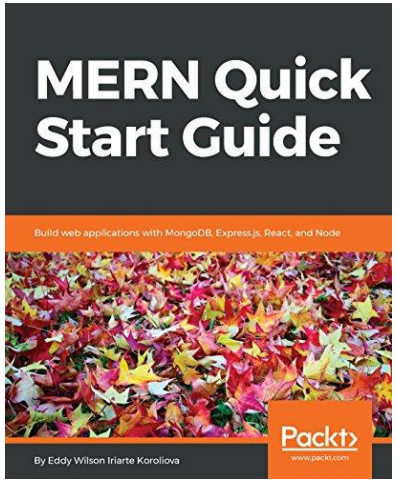

HTML
CSS
JS
Frontend
Backend
Database
Full Stack

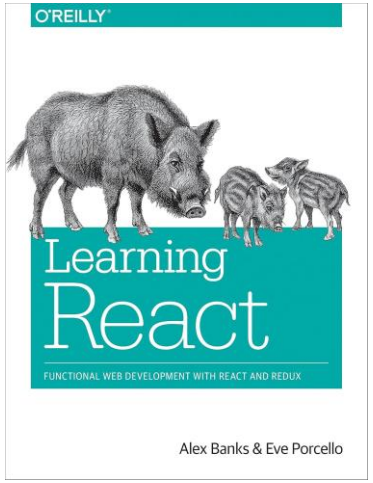
MERN Stack Development



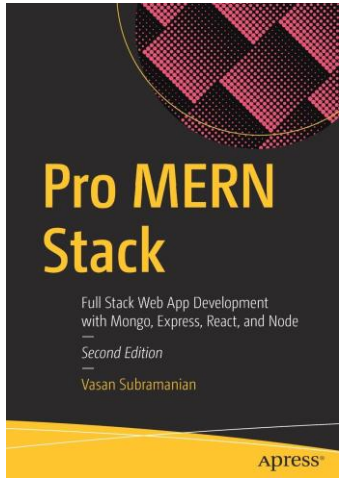
A Brain-Friendly Guide
Head First
JavaScript
O'REILLY
Michael Morrison

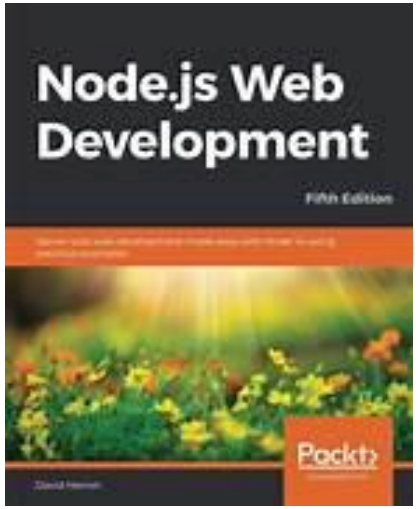
A Brain-Friendly Guide
Head First
HTML and CSS
O'REILLY

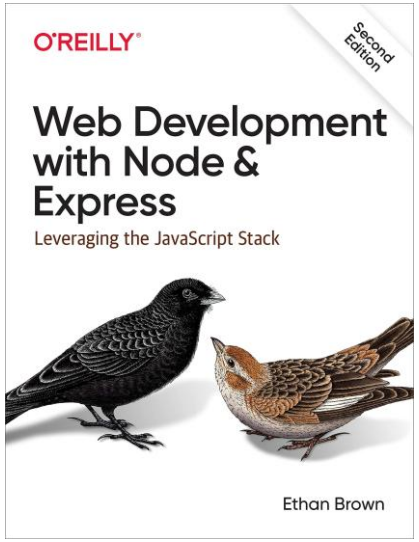

MERN Quick Start Guide
Build web applications with MongoDB, Express.js, React, and Node
Packt
By Eddy Wilson Iriarte Koroliova

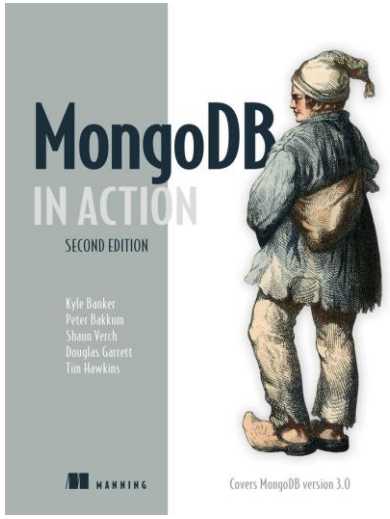

O'REILLY
Learning
React
FUNCTIONAL WEB DEVELOPMENT WITH REACT AND REDUX
Alex Banks & Eve Porcello

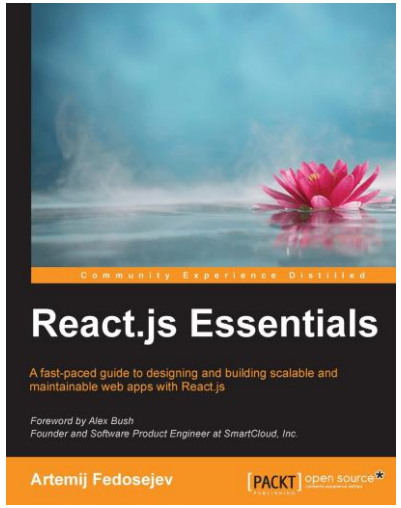
M E R N


Pro MERN Stack
Full Stack Web App Development with Mongo, Express, React, and Node
Second Edition
Vasan Subramanian
apress


Node.js Web Development
Fifth Edition
Packt


O'REILLY
Second Edition
Web Development with Node & Express
Leveraging the JavaScript Stack
Ethan Brown


MongoDB
IN ACTION
SECOND EDITION
Kyle Banker
Peter Bakkum
Shaun Verch
Douglas Garrett
Tim Hawkins
MANNING
Covers MongoDB version 3.0


Community Experience Distilled
React.js Essentials
A fast-paced guide to designing and building scalable and maintainable web apps with React.js
Foreword by Alex Bush
Founder and Software Product Engineer at SmartCloud, Inc.
Artemij Fedosejev
PACKT open source

Resources
&
References

Download & Install

Installing Visual Studio Code (VS Code)

- Go to the official website: **https://code.visualstudio.com/**
- Download the installer for your operating system:
  - Windows
  - macOS
  - Linux (.deb / .rpm)
- Install using default options
- Recommended extensions for MERN development:
  - **ESLint**
  - **Prettier – Code Formatter**
  - **JavaScript (ES6) Snippets**
  - **MongoDB for VS Code**
  - **Thunder Client / REST Client** (test APIs)
- Open VS Code and create your first workspace folder

Visual Studio Code

https://code.visualstudio.com/download

Download & Install

Installing Node.js

- Visit the official Node.js website: **https://nodejs.org**
- Download the **LTS (Recommended)** version for your OS
  - Windows
  - macOS
  - Linux
- Run the installer and follow default settings
- After installation, verify using terminal:
  - node -v
  - npm -v
- Node.js comes with **npm** (Node Package Manager)

  → used to install libraries like Express, React tools, etc.

https://nodejs.org/en/download

Download & Install

Installing MongoDB Compass

- Visit the official download page:

  https://www.mongodb.com/try/download/compass
- Select your operating system:
  - Windows (.msi)
  - macOS (.dmg)
  - Linux (.deb / .rpm)
- Download the **stable version**
- Install using the default settings
- Launch Compass and connect to the default local server:

  mongodb://localhost:27017

Features of MongoDB Compass:

- Visual database explorer
- Create, view, and edit collections
- Run queries visually
- Monitor schema and indexes

mongoDB Compass

https://www.mongodb.com/try/download/community

MERN Stack Learning Roadmap

Day 1: Overview of the MERN Stack and the roles of Node, Express, MongoDB, and React

Day 2: Modern JavaScript foundations essential for working with Node and React

Day 3: Introduction to **Node.js** and the fundamentals of server-side development

Day 4: Building structured backend services using **Express.js** routing and middleware

Day 5: Understanding **MongoDB** concepts and working with data in a NoSQL environment

Day 6: Integrating **Express** with **MongoDB** to design functional API workflows

MERN Stack Learning Roadmap

Day 7: Introduction to **React** and building UI components with a component-driven architecture

Day 8: Advancing in **React** with state management, hooks, and dynamic interactions

Day 9: Deep dive into **React** application logic, forms, and client-side data handling

Day 10: Connecting the frontend (React) with the backend (Node/Express) for full-stack integration

Day 11: Case Study 1: Analysis of a complete MERN application and its architecture

Day 12: Case Study 2: Exploration of an advanced MERN feature implementation

Day 1: Overview of the MERN Stack

- What is the MERN Stack?
- Full-stack workflow: frontend → backend → database
- Roles of **Node, Express, MongoDB, React**
- High-level MERN architecture
- Environment setup (Node.js, VS Code)
- Quick starter demo

Web Development Overview

Web development is divided into **three main layers**:

Frontend (Client-side)

- What the user sees (UI)
- Built with **HTML, CSS, JavaScript**
- Example: buttons, forms, pages

Backend (Server-side)

- Logic, processing, authentication
- Built with **Node.js, Express.js**
- Example: login API, database operations

Database

- Stores data
- In MERN: **MongoDB** (NoSQL)

What is Full-Stack Development?

A **Full-Stack Developer** works on:

Layer	Responsibilities
Frontend	UI, components, routing
Backend	APIs, business logic
Database	CRUD, schemas
Integration	Connect frontend ↔ backend

What is Full-Stack Development?

Full-Stack Workflow

User interacts with frontend (React)

Frontend sends request to backend

Backend processes (Node + Express)

Database stores or returns data (MongoDB)

Response goes back to UI

What is MERN?

MERN = MongoDB + Express + React + Node

Layer	Technology	Purpose
Frontend	React.js	UI & components
Backend	Express.js	API creation
Runtime	Node.js	JS on server
Database	MongoDB	Store data

1. **MongoDB:** NoSQL database
2. **Express.js:** Web API framework
3. **React.js:** Frontend UI library
4. **Node.js:** JavaScript runtime

Architecture

1.Client (React)

1. The user interacts with a React app running in the browser.
2. React handles UI state, routing (client-side), and calls backend APIs (usually REST) via fetch/axios.

2.Server (Node + Express)

1. Express defines API endpoints (e.g., GET /api/products, POST /api/auth/login).
2. Express middleware handles parsing, authentication, logging, error handling, and security concerns (CORS, rate limiting).
3. The server contains business logic, validation, and coordinates with the database.

3. Database (MongoDB)

1. Express / server code uses a MongoDB driver or ODM (e.g., Mongoose) to read and write JSON-like documents.
2. Data is persisted as flexible BSON/JSON documents, which map naturally to JavaScript objects used in Node.

4. Typical flow

1. Browser → (HTTP) → Express API → (query) → MongoDB → response → Browser.

Roles

Node.js (runtime)

- **Role**: Runs JavaScript on the server — enables full-stack JavaScript.
- **Responsibilities**:
  - Execute backend JavaScript code (controllers, services, background jobs).
  - Manage HTTP server (via frameworks like Express) and WebSocket servers.
  - Handle I/O-bound tasks efficiently using its non-blocking event loop (great for many concurrent connections).
- **Notes**:
  - Single-threaded by default; use worker threads or clustering for CPU-intensive tasks.
  - Rich ecosystem (npm) provides libraries for virtually everything.

Express.js (web framework)

Role: Lightweight web framework on top of Node for building APIs and server apps.

Responsibilities:

- Define routes and middleware (parsing JSON, authentication, file uploads).
- Handle request/response lifecycle and centralized error handling.
- Integrate with template engines or serve static files when necessary.

Why use it: Minimal, unopinionated, easy to compose middleware and scale. Works well with REST.

MongoDB (database)

Role: Document-oriented NoSQL database for storing application data.

Responsibilities:

- Persist JSON-like documents (flexible schema).
- Provide querying, indexing, aggregation, and replication for high availability.

Why use it:

- Schema flexibility allows rapid iteration on data models.
- Good fit for JavaScript apps since stored documents resemble JS objects.

Considerations:

- Not relational — joins are limited; you often model relations differently (embedding vs referencing).
- Use transactions for multi-document atomicity when needed (supported in modern MongoDB versions).

React (frontend)

Role: UI library for building component-based user interfaces.

Responsibilities:

- Render UI based on state and props.
- Handle client-side routing (React Router), forms, and local component state.
- Make API calls to the server and update views on response.

Why use it:

- Component reusability, fast DOM updates via virtual DOM, large ecosystem.

Why Do We Install Node.js?

- Node.js lets us **run JavaScript outside the browser**
- It acts like the *"engine room"* of a MERN application
- Example scenario:
  - You build a login form in **React**
  - React sends login details to **Node + Express**
  - Node checks the database (**MongoDB**)
  - Node sends back:
    - valid user → login success
    - invalid user → show error
- **Without Node.js, React cannot talk to the database.**

  Node is the "bridge" that connects frontend and backend.

Case Study: LinkedIn-Style App Using MERN

- Users can:
  - Create a professional profile
  - Add experience, skills, education
  - Connect with others
  - Make posts & share updates
  - Search people & jobs
- Tech stack:
  - **React** → UI (profile, feed, search, messages)
  - **Node + Express** → APIs, business logic
  - **MongoDB** → Users, connections, posts, messages

Profile Creation & Update

- User fills profile form in **React**:
  - Name, headline, about, skills, experience
- React sends data to backend:
  - POST /api/profile (new) or
  - PUT /api/profile (update)
- **Express + Node**:
  - Read user from token (logged-in user)
  - Validate data
  - Save/update profile in **MongoDB**

- MongoDB stores profile as a document:

```
{
  "_id": "u101",
  "name": “XYZ",
  "headline": "Full Stack Developer",
  "location": "Bangalore",
  "skills": ["React", "Node.js", "MongoDB"],
  "experience": [
    { "company": "ABC Tech", "role": "Developer", "years": 2 }
  ]
}
```

- React reloads profile page with updated data

Connections (Like “Connect” on LinkedIn)

- On another user’s profile, user clicks **Connect**
- React sends:
  - POST /api/users/:id/connect
- **Express + Node**:
  - Check both users exist
  - Add a connection entry into MongoDB:
    - Current user’s connections
    - Target user’s connections

- MongoDB:

```
{
  "_id": "u101",
  "connections": ["u202", "u303", ...]
}
```

- Later, this connections list is used to:
  - Build the **feed**
  - Show **mutual connections**

Feed: Professional Posts & Updates

- Users share posts like:
  - “Started a new role…”
  - “Uploaded a project…”
  - “Looking for opportunities…”
- React calls:
  - GET /api/feed
- **Express + Node**:
  - Find all connections of the current user
  - Fetch posts from those users from **MongoDB**
  - Sort by time (newest first)

- MongoDB post document example:


```
{
  "_id": "post501",
  "userId": "u202",
  "text": "Excited to join XYZ as a Senior Engineer!",
  "likes": ["u101", "u303"],
  "comments": [],
  "createdAt": "2025-01-21T09:00:00Z"
}
```


- React displays a feed similar to LinkedIn Home

Reactions, Comments & Engagement

- User clicks **Like**:
  - React → PATCH /api/posts/:id/like
- **Express + Node**:
  - Add/remove user from likes array
- MongoDB updates:

  "likes": ["u101", "u303", "u404"]
- React updates like count in the UI
- **Comments:**
- React sends:
  - POST /api/posts/:id/comments
- Express saves comment in MongoDB
- React updates the comments section under the post

Job Posts & Applications

- Companies or recruiters create job posts:
  - POST /api/jobs
- MongoDB stores:

```
{
  "_id": "job999",
  "title": "MERN Stack Developer",
  "company": "TechCorp",
  "location": "Remote",
  "skills": ["React", "Node.js", "MongoDB"],
  "applications": []
}
```

- When a user clicks **Apply**:
  - React → POST /api/jobs/:id/apply
- **Express + Node**:
  - Add user ID to applications array in MongoDB
- Recruiter sees list of applicants in a dashboard (React UI)

Why MERN is a Good Fit for LinkedIn-Style Apps

- **React**
  - Rich UI: feed, profiles, messaging pane, search
  - Fast updates and client-side routing
- **Node + Express**
  - Handles many user actions: connect, follow, like, post, apply
  - Easy to build REST APIs and auth flows
- **MongoDB**
  - Flexible schema for profiles, posts, connections, jobs
  - Easy to store nested documents (experience, comments, skills)

Typical folder structure

```
project-root/
├─ client/ # React app (create-react-app / Vite)
│  ├─ src/
│  └─ package.json
├─ server/ # Node + Express backend
│  ├─ src/
│  │  ├─ controllers/
│  │  ├─ routes/
│  │  ├─ models/ # Mongoose schemas or data layer
│  │  ├─ services/
│  │  └─ app.js
│  └─ package.json
├─ .env # environment variables for local dev
└─ README.md
```

Fundamentals with Basic JavaScript

What is JavaScript?

- Programming language of the web
- Runs in browser + server (Node.js)
- Used in **entire** MERN stack (frontend + backend + DB queries)
- JavaScript is the **only language used across the entire MERN stack**

  → React (frontend)

  → Node.js + Express (backend)

  → MongoDB (JSON documents that resemble JS objects)
- Unified language = faster development, easier debugging
- Modern JS (ES6+) makes code cleaner, modular, scalable

JavaScript Syntax Overview

- JavaScript is **case-sensitive** — userName and username are treated as different variables.
- Code blocks such as functions, loops, and conditionals use curly braces **{ }**.
- Semicolons are optional, but recommended for clarity and avoiding automatic semicolon insertion issues.
- **CamelCase** naming (e.g., firstName, totalMarks) is the standard for variables and functions.

Variables in JavaScript (let, const)

- **let** is used for variables whose values need to change during execution (e.g., counters, form inputs).
- **const** is used for variables that should not be reassigned, such as configuration values or fixed data.
- Both let and const are block-scoped, meaning they exist only within the { } block they are declared in.
- **var** should be avoided in modern JavaScript because it has function-level scope and can lead to unexpected behavior.

```
// Case-sensitivity + block usage
let studentName = "Riya";

// Different variable (case-sensitive)
let StudentName = "Priya";

if (true) {
  let message = "Inside block";
  console.log(message);
}

// console.log(message);
Error: block scoped
```

```
// Variables (let, const)

let marks = 85;   // can change

marks = 90;       // allowed


const PI = 3.14;  // cannot change

// PI = 3.18;  Error


// var example (not recommended)

var city = "Chennai";
```

Data Types in JavaScript

Primitive Types

- **string**: text values — "Hello"
- **number**: integers or decimals — 40, 3.14
- **boolean**: logical values — true, false
- **null**: intentional empty value
- **undefined**: declared but not assigned

Reference Types

- **Object**: key–value pairs used everywhere in MERN (Express responses, MongoDB documents).
- **Array**: ordered list of data used heavily in React to render lists and handle multiple values.

```
// Data Types
// Primitive types
let name = “ABC";          // string
let age = 20;              // number
let pass = true;           // boolean
let value = null;          // null
let data;                  // undefined
```

```
// Reference Types
let student = { name: “XYZ", dept: "CSE" };  // object
let skills = ["HTML", "CSS", "JavaScript"];   // array
```

Objects in JavaScript

- Objects are collections of key-value pairs and represent real-world entities like students, products, or user profiles.
- They are essential in MERN because Express handles request and response bodies as objects.
- MongoDB stores entire documents in object format, making them natural to work with in JavaScript applications.

Arrays in JavaScript

- Arrays store lists of items and are fundamental for handling collections of data such as posts, comments, users, or skills.
- Arrays form the basis for rendering UI lists in React (using .map).
- Many MERN operations involve array manipulation, making them an essential concept.

```
// Objects
let product = {
  title: "Laptop",
  price: 55000,
  inStock: true
};

console.log(product.title);   // Laptop
console.log(product.price);   // 55000
```

```
// Arrays
let languages = ["JS", "Python", "C++"];

console.log(languages[0]);
// JS
languages.push("React");
// add new item
console.log(languages);
// ["JS", "Python", "C++", "React"]
```

Functions in JavaScript

- Functions are reusable blocks of code that take inputs, perform operations, and return outputs.
- They are central to MERN because React components, Express route handlers, and database operations all use functions.
- JavaScript supports multiple function styles, including traditional functions and function expressions.

Arrow Functions (Modern ES6 Syntax)

- Arrow functions provide a cleaner and more concise way to write functions.
- They eliminate the need for the function keyword, making code shorter and easier to understand.
- They also inherit the surrounding this context, reducing common bugs in callback-based code.
- Arrow functions are widely used in React for event handlers and component logic.

```
// Standard Function
function greet() {
  return "Hello everyone!";
}

console.log(greet());
```

```
// Function with Parameters
function add(a, b) {
  return a + b;
}

console.log(add(10, 20)); // 30
```

```
// Arrow Function
const square = (n) => n * n;
console.log(square(5)); // 25

const sayHello = name => `Hello ${name}!`;
console.log(sayHello("ABC")); // Hello ABC!
```

Template Literals (Backtick Strings)

- Template literals allow string interpolation using ${ }, making strings easier to read and construct.
- They are extremely useful for printing formatted data, building dynamic messages, and generating HTML snippets.
- Backticks support multi-line strings, which improves readability in UI and backend logging.

```
let firstName = "Ravi";
let course = "MERN";

console.log(`My name is ${firstName} and I am learning ${course}.`);
```

```
let description = `
This is a MERN course.
Day 1: JavaScript Essentials.
`;
console.log(description);
```

Case Study: Simple E-Commerce Product CRUD with JavaScript

- ✓ Build a small **product management system** using JavaScript
- ✓ Implement basic **CRUD operations**: Create, Read, Update, Delete
- ✓ Use **arrays, objects, functions, and loops**
- ✓ Run everything in the **console** (no UI needed for now)

Problem Statement: Product Management in a Store

- We manage a **list of products** in a small online store
- Each product has:
  - id
  - name
  - price
  - inStock
- We want to:
  - Add a new product
  - View all products
  - Update a product
  - Delete a product

Data Structure Design - Representing Products in JavaScript

- Each product is an **object**
- All products are stored in an **array**
- This array acts as our **in-memory database**

```
// Our "database"
let products = [
  { id: 1, name: "Laptop", price: 55000, inStock: true },
  { id: 2, name: "Mouse", price: 800, inStock: true },
  { id: 3, name: "Keyboard", price: 1500, inStock: false }
];

console.log("Initial Products:", products);
```

CREATE: Add a New Product

Create Operation (C in CRUD)

- We create a function addProduct
- It:
  - Accepts product details as parameters
  - Calculates a new id
  - Pushes a new object into the products array

```
// Example usage
addProduct("Headphones", 2000, true);
console.log("After CREATE:", products);
```

```
function addProduct(name, price, inStock) {
  const newId = products.length > 0
    ? products[products.length - 1].id + 1  : 1;
  const newProduct = {
    id: newId,
    name: name,
    price: price,
    inStock: inStock
  };
  products.push(newProduct);
  console.log("Product added:", newProduct);
}
```

READ: List All Products (with Loop)

Read Operation (R in CRUD) – View All Products

- We create a function listProducts
- Use a **for loop** to print each product
- Shows how loops work with arrays

```
// Example usage
listProducts();
```

```
function listProducts() {
  console.log("Product List:");
  for (let i = 0; i < products.length; i++) {
    const p = products[i];
    console.log(
      `ID: ${p.id}, Name: ${p.name}, Price: ₹${p.price},
In Stock: ${p.inStock}`
    );
  }
}
```

READ: Find a Product by ID

Read Operation – Find Product by ID

- We create a function getProductById
- It:
  - Loops through products
  - Returns the product that matches the given id

```
// Example usage
const found = getProductById(2);
console.log("Found product:", found);
```

```
function getProductById(id) {
  for (let i = 0; i < products.length; i++) {
    if (products[i].id === id) {
      return products[i];
    }
  }
  return null; // if not found
}
```

UPDATE: Change Product Details

Update Operation (U in CRUD)

- We create updateProduct
- It:
  - Finds product by id using a loop
  - Updates fields like name, price, or inStock if provided
- Demonstrates:
  - Conditional update
  - Objects + loops

DELETE: Remove a Product

Delete Operation (D in CRUD)

- We create deleteProduct
- It:
  - Finds index using a loop
  - Removes the product using splice
- Shows:
  - Looping with index
  - Modifying the array

```
function updateProduct(id, newName, newPrice, newInStock) {
  for (let i = 0; i < products.length; i++) {
    if (products[i].id === id) {
      if (newName !== undefined) {
        products[i].name = newName;
      }
      if (newPrice !== undefined) {
        products[i].price = newPrice;
      }
      if (newInStock !== undefined) {
        products[i].inStock = newInStock;
      }
console.log("Product updated:", products[i]);
      return;
    }
  }
  console.log("Product not found with id:", id);
}

// Example usage:
updateProduct(2, "Wireless Mouse", 1200, true);
console.log("After UPDATE:");
listProducts();
```

```
function deleteProduct(id) {
  for (let i = 0; i < products.length; i++) {
    if (products[i].id === id) {
      const removed = products.splice(i, 1); // removes 1 item at index i
      console.log("Deleted product:", removed[0]);
      return;
    }
  }
  console.log("Product not found with id:", id);
}
```

```
// Example usage
deleteProduct(3);
console.log("After DELETE:");
listProducts();
```

```
console.log("Initial products:");
listProducts();

console.log("\n--- Adding a new product ---");
addProduct("Smartphone", 25000, true);
listProducts();

console.log("\n--- Updating product with id 1 ---");
updateProduct(1, "Gaming Laptop", 75000, true);
listProducts();

console.log("\n--- Deleting product with id 2 ---");
deleteProduct(2);
listProducts();
```

Learning Outcomes

What Did We Learn from This Case Study?

- How to **model data** using arrays and objects
- How to build **CRUD operations** using basic JavaScript
- How to use **loops** to search, list, and update data
- How simple code structure can become the base for a real backend later
- This logic can later be moved into **Node.js + Express** and connected to **MongoDB**

Explore as Home Fun…

- **Create a JavaScript object** representing a product (for example: name, price, brand, inStock) and print all its properties using template literals.
- **Write a function** that takes a student's marks in 3 subjects as parameters and returns both the total and the average marks.
- **Create an array** of 5 favorite movies and print each movie name using a loop.
- **Write an arrow function** that takes a number as input and prints whether it is even or odd.
- **Create a "profile card" string** using template literals that includes your name, college, course, and at least two hobbies, then print it.
- **Create a timetable object** where each key is a weekday (e.g., "Monday", "Tuesday") and the value is an array of subjects for that day. Print the subjects for any one chosen day.

Spotify CRUD Task:

Create an array of song objects to simulate a simple Spotify playlist. Each song should have properties like id, title, artist, and duration. Using basic JavaScript:

- Add a new song to the playlist (Create)
- Display all songs using a loop (Read)
- Update the title or artist of a specific song using its id (Update)
- Remove a song from the playlist using its id (Delete)

Day 1 Recap – JavaScript Foundations

MERN Stack Overview

- Structure and flow of a full-stack MERN application
- Roles of Node.js, Express.js, MongoDB, and React
- Importance of JavaScript as the common language across all layers

Core JavaScript Fundamentals

- Variable declarations using **let** and **const**
- Primitive and reference data types
- Object and array representations
- Function declarations and modern **arrow functions**
- String formatting using **template literals**

Practical Foundations

- Basic console-based programming exercises
- Object and array manipulation
- Simple function creation and usage
- Introductory CRUD logic through an e-commerce example

Outcome

- Establishes the fundamental JavaScript skills needed for progressing into advanced ES6 features, asynchronous programming, and full-stack development in upcoming sessions.

Scenario-Based Brainstormer

A new food-delivery startup wants to show customers a live-updating list of available dishes based on time of day (breakfast → lunch → dinner).

Question:

How could JavaScript concepts like arrays, objects, functions, and template literals be used to dynamically change the menu displayed to the user throughout the day?

Food-delivery startup → menu changes based on time of day:

- Morning → breakfast items
- Afternoon → lunch
- Night → dinner

We'll use:

- **Objects** to model dishes
- **Arrays** to group dishes by meal
- **Functions** to pick the right menu
- **Template literals + loops** to "display" it